

第084课 实验册：写出并审计一个标量自动微分器

实验任务

你将逐步核验加法、乘法、共享节点、前向模式、反向模式与有限差分，最后把相同机制用于一个ReLU神经元。核心代码不导入PyTorch、JAX或NumPy；因此它不是把现成框架包装成“从零实现”。目标是理解依赖图与链式法则，功能边界见README.md。

建议用90至120分钟完成，其中至少一半时间用于运行前手算与失败原因解释。不要先翻answers.md。遇到结果不符时保留失败记录，不要直接把测试期望改成程序现有输出。

0 可重复运行与文件职责

所有命令在本目录执行。Python 3.10以上即可运行核心实验，交付实测为3.12.14。data/gradient-cases.csv给出预先指定的5个核验点；它们不是训练样本，没有拟合/选模流程，也不需要人为拆出“测试集”。

```
python generate_data.py --directory outputs/generated-data
python experiment.py --out outputs/result.json
python test_experiment.py --out outputs/test-result.json
python -O test_experiment.py --out outputs/test-result-optimized.json
```

两种解释器应各执行19项unittest，0失败、0错误。常规结果与优化结果中的optimized字段应不同。原始数据哈希应一致；若不同，首先检查你是否编辑过CSV，不要立刻把数值差异归因于操作系统。

- autodiff.py: Value反向模式、Dual前向模式、中心差分
- experiment.py: 确定性测试场景与结果汇总
- test_experiment.py: 真正的unittest测试和退出码
- experiment-result.json: 交付环境冻结实测，不是预先手填答案
- plots.py: 四幅原创图的可重建脚本

A 只算一个操作：把局部规则和总导数分开

先写出加法 $c = a + b$ 与乘法 $c = ab$ 的两个局部偏导。令 $a=2$ 、 $b=3$ ，预测函数值和两项梯度，再运行。

```
from autodiff import Value
for operator in ["add", "mul"]:
    a, b = Value(2), Value(3)
    out = a+b if operator == "add" else a*b
    out.backward()
    print(operator, out.data, a.grad, b.grad)
```

现在把`out.backward()`改成`out.backward(seed=2)`。预测哪几个数翻倍、哪个不变。检查种子改变的是最终输出组合的敏感度，并不会把前向函数值变成两倍。

进一步检查 $x = 3, z = x * x$ 。如果梯度是3而不是6，优先检查是否把两个相同父节点的输入槽位误删了。图中的节点身份可以相同，乘法规则仍有左右两项。

B 核心手算：共享节点不得覆盖

设 $x=2, y=3, t=xy, q=tt, z=q+t$ 。纸上画图，并为每个节点分别填写前向值、已经接收到的梯度、尚未处理的下游节点。严格按 z, q, t 的顺序反传。

```
from autodiff import Value
x, y = Value(2, label="x"), Value(3, label="y")
t = x*y; t.label="t"
q = t*t; q.label="q"
z = q+t; z.label="z"
order = z.backward()
for node in order:
    print(node.label, node.data, node.grad, len(node.edges))
```

验收：必须能解释为什么`t.grad`是13，而不是12、6或1；能区分图有5个节点与6条输入边；能说明为什么`t`要等待来自`q`的贡献后才继续向`x`和`y`传播。只背出39和26而说不出来源，不算通过。

可选错误实验：在你自己的临时副本中，把`parent.grad +=`改成覆盖赋值，观察共享图失败；完成后恢复原文并重跑全部测试。不要覆盖冻结实测结果来掩盖故意注入的错误，也不要将错误副本放入交付代码路径。

C 读实现：节点去重，边不能盲目去重

阅读`Value.topology()`，找出显式`stack`、`seen`与`expanded`标记分别解决什么问题。手画一个菱形依赖：`x`同时用于两个中间节点，两节点再相加。检查顺序能否保证所有父节点先于子节点出现。

在本实现中，每次`backward`都会把当前输出可达节点的`grad`清零。请连续调用两次，并记录`x.grad`没有翻倍。随后换`seed=2`再次调用。把这条语义写入实验报告，旁边注明：不能直接套用到PyTorch叶子`.grad`的默认跨调用累积行为。

```
x = Value(3)
y = x*x+x
for seed in [1, 1, 2, 0]:
    y.backward(seed)
    print(seed, y.data, x.grad)
```

本课禁止修改`Value`内部`_data`或`edges`。要在新输入处求导，应创建新图。正常`data`属性只读，尝试赋值应触发`AttributeError`；这能阻止最直接的过期前向值使用，但不是完整的生产级原地修改检测系统。

D 四路核验一个平滑复合函数

函数为 $f(x, y) = (xy + \sin x)e^y$ 。先独立推导两个偏导，尤其注意对 y 求导时乘积两边都变化。之后比较Value反向、Dual前向、独立解析式与中心差分。

```
import math
from autodiff import Value, Dual, finite_difference
from experiment import composite, composite_float
x, y = Value(.7), Value(-1.2)
out = composite(x, y); out.backward()
print("reverse", out.data, x.grad, y.grad)
print("forward dx", composite(Dual(.7, 1), Dual(-1.2, 0)).tangent)
print("forward dy", composite(Dual(.7, 0), Dual(-1.2, 1)).tangent)
print("finite", finite_difference(composite_float, [.7, -1.2], 1e-5))
```

验收：报告每条路线的数值与差异，不只打印True。查看CSV的另外四个点，说明它们覆盖哪些符号或特殊情况。不要在看过错误后只保留恰巧通过的点；添加新用例时应保留原用例与修正说明。

E 差分步长扫描与不可微点

查看结果中的finite_difference_sweep或打开图3。找出误差最小的已试步长；观察更小步长后误差是否继续下降。记录这是“这次函数、点、精度与扫描范围”下的结果，不是所有梯度核验通用的最佳步长。

然后检查ReLU在0处：实现选定的反向规则是0，而中心差分为0.5。写出左右差分的分子与分母，解释为什么无论缩小多少，中心差分仍给0.5。若仅以二者不一致为由改代码成0.5，你改变的是不可微点约定，而不是证明原实现偏离了一个存在的导数。

```
from autodiff import Value, finite_difference
x = Value(0); out = x.relu(); out.backward()
print("chosen rule", x.grad)
for h in [1e-1, 1e-3, 1e-5]:
    print(h, finite_difference(lambda v: max(0., v), [0.], h))
```

F 前向与反向不是同一个矩阵乘积

令 $f = (xy, x^2 + y)$ ，在(2,3)手算完整2乘2 Jacobian。前向输入方向 $v=(1,-1)$ ，反向输出权重 $u=(2,-1)$ 。先分别计算 Jv 与 J 的转置乘 u ，再运行：

```
from autodiff import Value, Dual
from experiment import vector_function
print("JVP", [a.tangent for a in vector_function(Dual(2, 1), Dual(3, -1))])
x, y = Value(2), Value(3)
f1, f2 = vector_function(x, y)
loss = 2*f1-f2; loss.backward()
print("VJP as column", x.grad, y.grad)
```

验收：解释 v 为什么有输入维数，而 u 为什么有输出维数。若输出扩展为三个分量，哪个种子需要三个元素？一次反向只返回当前 u 对应的乘积，怎样改变 u 才能逐行恢复整个Jacobian？

G 一个神经元的端到端微分

固定输入(2,-1)，权重(0.4,-0.8)，偏置0.1，ReLU输出，目标1，半平方损失。依次手算预激活、预测、误差、损失，以及对三个参数的导数。再检查result.json的neuron字段。

不要把梯度直接当作更新后的参数。本实验没有选学习率、没有执行优化，也没有评估新样本。如果你可选地尝试一步更新，应新建图并说明步长，而不能在旧图上改数据后重复调用backward，期待它自动重算前向。

H 图像与异常行为复核

```
python plots.py --directory outputs/figures
```

图1检查共享路径，图2检查贡献相加，图3检查差分边界，图4检查乘积方向。核对每张PNG与SVG是否齐全且中文可读。代码拒绝log(0)、非有限Value、非正整数幂等未支持输入；异常本身是明确API边界，不应通过静默输出0“修复”。

课后习题

1. 对 $x = 3, z = x * x + x$ ，手算前向值、全部路径贡献与梯度。若只保留一条x到乘法的贡献会怎样？
2. 对共享图 $t = xy, z = t^2 + t$ ，在x=2、y=3逐节点计算反向梯度，并独立展开验证。
3. 为何拓扑遍历按对象身份去重，而不能按data数值去重？构造两个数值相同但应分别保留的叶节点。
4. 本课连续两次backward为何不是累加？写出seed=2的含义，并比较PyTorch默认叶子梯度语义。
5. 推导 $(xy + \sin x)e^y$ 两个偏导，指出对y求导最容易漏掉哪项。
6. 对 $f = (xy, x^2 + y)$ 在(2,3)计算J、Jv与 $J^\top u$ ，v=(1,-1)，u=(2,-1)。解释二者种子所在空间。
7. ReLU在0处中心差分为何为0.5？能否说明真实导数为0.5？如何设计一个不跨拐点的正区间差分测试？
8. 有一百万参数、一个标量损失，需要全部梯度；与一个输入、很多输出需要整条导数向量的情况相比，哪种模式更合适，为什么？
9. 手算本课ReLU神经元的损失与参数梯度；若预激活严格为负会发生什么？
10. 有同学把Value数据改成新参数后直接在旧图backward，又有同学把步长设为 10^{-15} 当作“最高精度核验”。分别解释风险并提出正确流程。

最终提交

保留实践JSON、两种解释器测试结果、共享图逐节点计算、四路导数对照、步长误差解释、ReLU不可微说明和JVP/VJP手算。最后写一段限制：当前引擎没有张量广播、GPU、高阶图或生产级数值稳定保障；这些限制不会因19项测试全绿而自动消失。